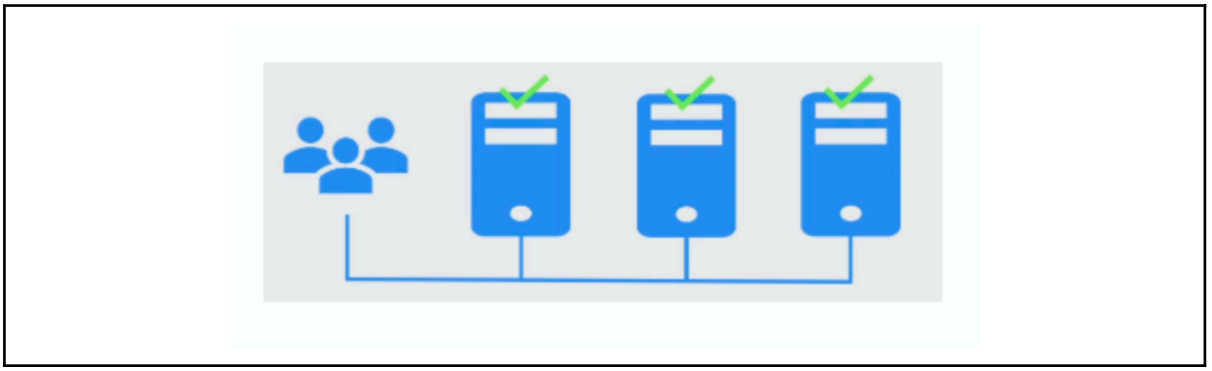
Section 7: System Performance

High Availability, Fault Tolerance & Failover

In this lecture, we will explore how modern systems stay available despite failures by using high availability, fault tolerance, redundancy, and intelligent failover strategies to deliver resilient, production-ready architecture.

Redundancy & Redundancy Strategies



Redundancy is one of the fundamental techniques for building reliable systems. The idea is simple, never allow a single component failure to bring down the entire application. If every crucial part has a backup that can seamlessly takeover, your system becomes far more resilient to failures. Redundancy can exist at multiple layers.

- ➔ At the **infrastructure level**, hardware redundancy means running multiple server or storage devices. So a hardware failure doesn't interrupt the service.

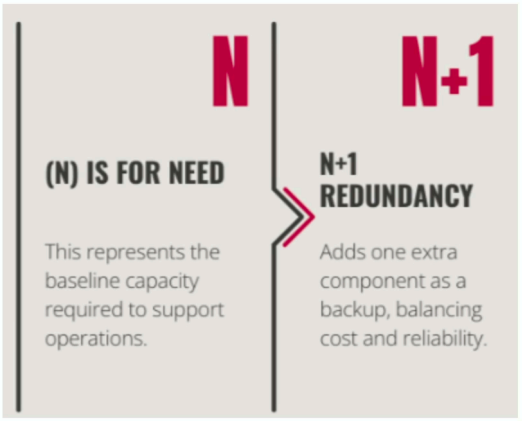
- ➔ At the **network layer** paths, routers, or availability zones ensure that connectivity is maintained even if the part of the network becomes unavailable.

- ➔ At the **application layer**, service redundancy involves running multiple instances of microservices or replicated databases so that requests can continue to be served even when individual instances fail.

However, simply duplicating components is not enough. Redundant systems must stay healthy, synchronized, and ready to take over automatically when failures occur. Otherwise your backup becomes just another component that may fail when you need it most.

As we move forward in this lecture, we will explore different redundancy strategies, and the trade-offs involved. Because every layer of redundancy improves availability but it also introduces additional cost, operational complexity, and synchronization challenges that architects must very carefully balance.

N + 1, Active-Active vs. Active-Passive



As systems become more critical, simply adding redundant components is not enough. You also need a strategy for how these components are used when failure happens. This is where N plus one, active-active,and active-passive architectures come into play. Each improves reliability, but they optimize for different trade-offs.

- ➔ **N + 1 Redundancy:** The principle here is simple, always keep one extra instance beyond your minimum required capacity. If your application needs two servers to handle peak traffic, you deploy three. That extra instance is not there to increase performance. It is there to absorb failures without impacting the users. This approach is common in infrastructure, databases, and even networking equipment.

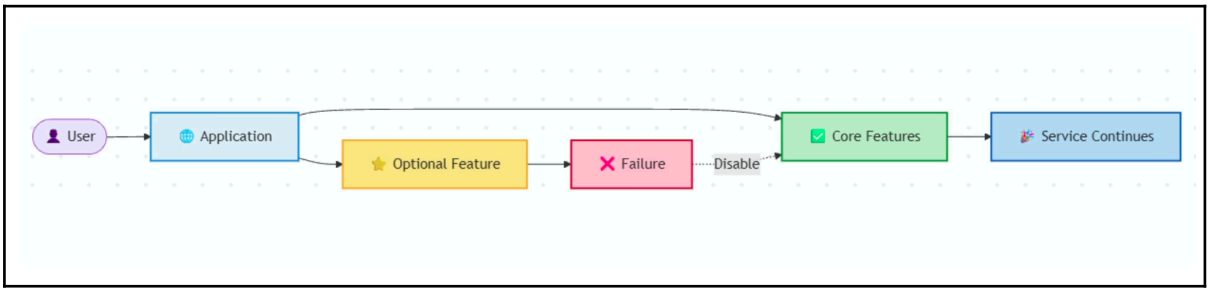
- ➔ **Active-Active:** Here, multiple nodes serve requests simultaneously with a load balancer distributing traffic across them. If one node fails, traffic is simply routed to the remaining healthy nodes, often without users even noticing. The trade-off is greater operational complexity, since data sessions, and state may need to remain synchronized across multiple active instances.

- ➔ **Active-passive:** Active-passive architecture keeps only one node actively serving traffic, while another waits in standby. If the active node fails, the standby is promoted and begins serving the request. This model is simpler to operate, and it is widely used for disaster recovery and databases. But the standby capacity remains mostly idle, failover typically introduces small recovery delay here.

There is not a universally best strategy. Active-active maximizes availability and resource utilization. Active-passive prioritizes simplicity, and n plus one ensures that your system has enough spare capacity to survive the failures. Choosing between them is an architectural decision based on your availability requirements, cost constraints, and the operational complexity that you tolerate.

Graceful Degradation

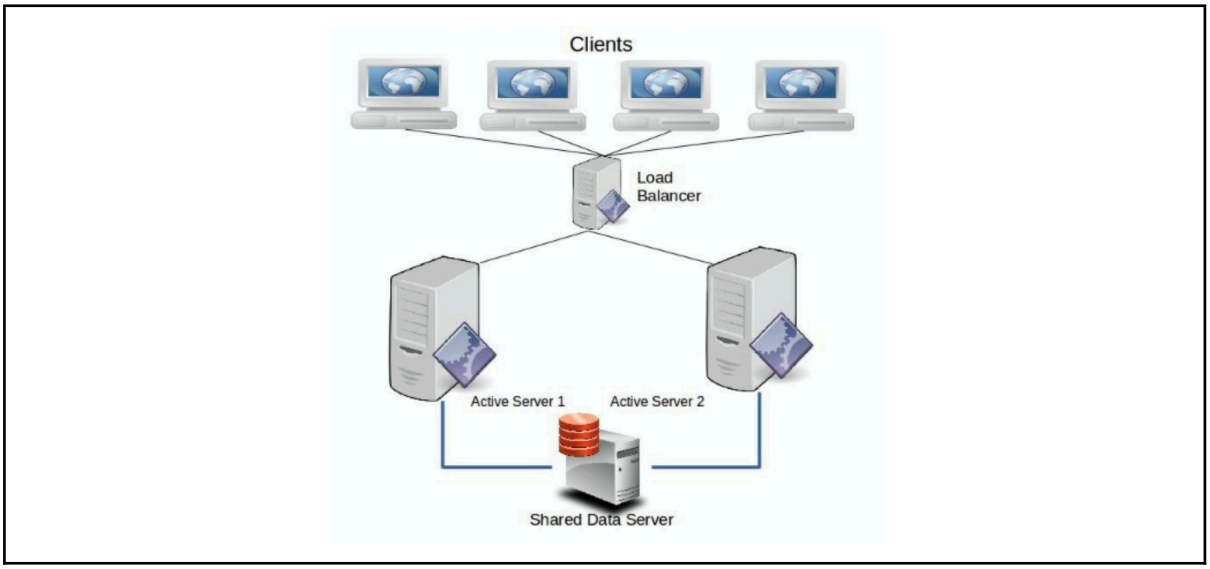
Failures don’t always have to result in down time. In many production systems, the better strategy is to keep the most important functionality running while temporarily sacrificing less critical features. This design principle is known as graceful degradation.



The idea is simple, when resources become constrained or a dependency fails, the system intentionally reduces functionality instead of falling completely. Users may notice that some features are unavailable, but they can still accomplish the primary task they came for. Consider an e-commerce platform during black Friday sales. Suppose the recommendation service becomes overloaded or unavailable. Rather than allowing that failure to impact the entire website, the application simply hides the recommendation section while keeping the product browsing, shopping cart, and checkout fully operational. From a business perspective, that is exactly the right trade-off. You protect the revenue generation path while temporarily disabling the non-essential feature.

Graceful degradation also improves user trust. A partially functional application is almost always better than an application that is completely unavailable. Achieving this requires architects to clearly identify which features are mission-critical and which can be safely disabled during failures. In modern distributed systems, graceful degradation is not an afterthought. It is designed into the architecture from the beginning itself. The goal is not to make failures invisible, but to ensure that they have the smallest possible impact on users while the system continues delivering its core values.

High Availability Patterns in Real-World Systems



High availability is not achieved by a single technology. It is the result of multiple architectural patterns working together to eliminate a single point of failure.Three of the most fundamental patterns are load balancing, replication, and failover.

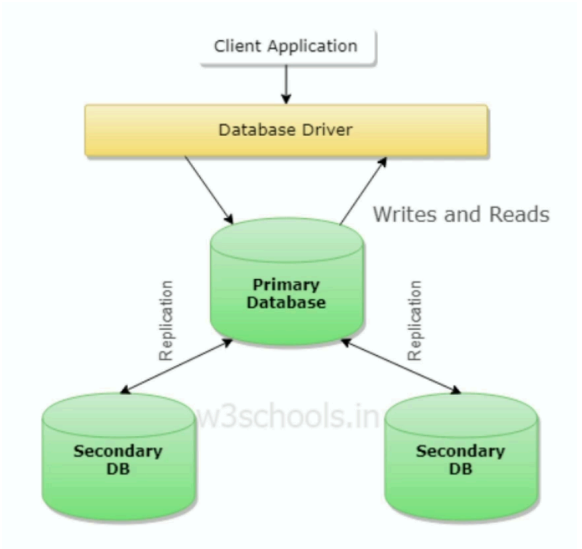
- **Load Balancer:** Their primary responsibility is to distribute incoming requests across multiple healthy instances, preventing any single server from becoming a bottleneck. Just as importantly, they continuously monitor instance health. If a server becomes unavailable, it is automatically removed from the pool and the traffic is redirected to healthy nodes. This makes load balancer a key building block for active-active architecture.

- **Replication:** Infrastructure can be replaced, but the data is far more valuable. By maintaining copies of data across multiple nodes or even multiple regions, systems can continue serving requests despite hardware failure or data center outages. Whether replication is synchronous or asynchronous depends on the required balance between consistency, latency, and availability.

- **Failover:** While replication ensures that a backup exists, failover ensures that the backup is actually used when it is needed. Health checks detect failure, and traffic or workloads are automatically transferred to a standby instance with minimal manual intervention. The faster and more reliable this process is, the smaller the impact on users.

In practice, these patterns rarely operate in isolation. A highly available production system typically combines load balancing for traffic distribution, replication for data resilience, and automated failover for rapid recovery. Together, they form the foundation of resilient systems that continue operating even when individual components fail, and they will inevitably fail.

Designing for Redundancy



Designing for redundancy is not about adding backup components at the end of the project. It is about assuming that failures will happen and ensuring the system continues operating when they do. Experienced architects design for failures from day one and not after the first outage.

- ➔ The first principle is **redundant components**. Every critical part of your architecture, whether it is web server, database, cache, or message broker, should have multiple instances. This eliminates a single point of failure and allows the system to continue serving requests even when individual components become unavailable.

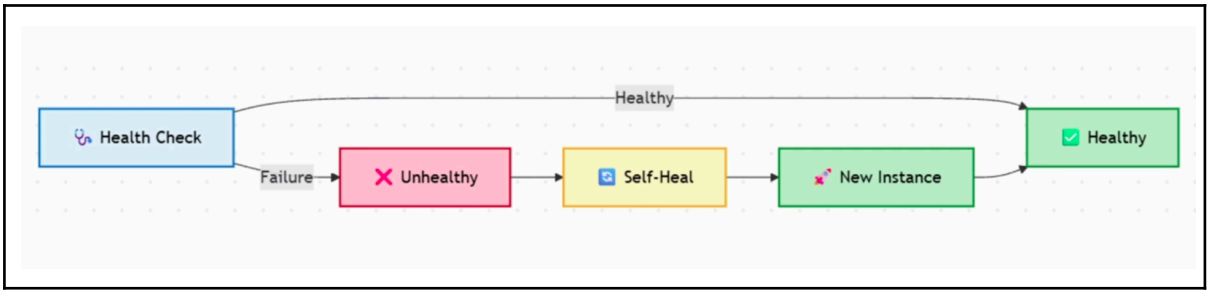
- ➔ The second principle is **geographical redundancy**. Multiple servers within the same data center protect you from hardware failure, but they don't protect you from a regional outage. By deploying applications and replicating data across multiple cloud regions or data centers, you can survive large-scale incidents such as power failures, network disruptions, or natural disasters. This is the cornerstone of disaster recovery planning.

- ➔ The final principle is **automated failover**. Redundancy is only effective if recovery happens automatically. Automated failovers use health checks and monitoring to detect failures and immediately redirect traffic to promote standby resources without waiting for any human intervention. The faster the system recovers, the lower the impact on the user.

Ultimately, redundancy is all about building systems that expect components to fail, and when combined with geographic distribution and automated failovers, failures become routine operational events rather than business-critical outages. And that is the mindset behind resilient and production-grade system design.

Health Monitoring & Self-Healing Systems

Building redundant systems is only half the challenge. You also need to know when something has failed and recover from it as quickly as possible. That is why health monitoring and self-healing have become essential capabilities in modern distributed systems.



Health monitoring continuously evaluates the condition of your infrastructure and services. Rather than waiting for users to report the problem, the system tracks metrics such as response times, error rate, resource utilization, and service availability. When these metrics cross the predefined threshold, alerts are generated so issues can be investigated before they become major outages. In production effective monitoring is often the first indication that something is going wrong.

Self-healing is the next step. Instead of relying on engineers to manually recover failed components, the platform automatically takes the corrective action. A failed container can be restarted, an unhealthy virtual machine can be replaced, or traffic can be redirected away from an unhealthy node. Platforms like kubernetes and cloud providers perform many of these recovery actions automatically and dramatically reduce the recovery time.

This reflects an important shift in modern architecture. We no longer assume that infrastructure is always reliable. Instead, we expect failures to occur and build systems that can detect, isolate, and recover from them with minimal human intervention. This combination of health monitoring and self-healing transforms reliability from a reactive process into a proactive process. Monitoring tells you when the system is unhealthy. While self-healing ensures that many failures are resolved automatically before your users even notice them.

Summary & Key Takeaways

- High Availability ensures systems remain operational even in the event of failures through redundancy and failover.
- Fault Tolerance involves designing systems to continue functioning despite partial failures, using strategies like N+1 redundancy and graceful degradation.
- Failover systems provide automatic switching to backup systems to maintain service availability.
- Load Balancers play a crucial role in distributing traffic and ensuring HA across servers.
- Health Monitoring and Self-Healing Systems are critical for ensuring that systems can detect failures and recover autonomously.
- Designing systems with redundancy and resilience is essential to ensure availability and performance.
- What's next:
    - Backup & Recovery Strategies

Interview Questions

- What is High Availability (HA) and why is it important in system design?
- Describe the difference between active-active and active-passive redundancy. When would you choose one over the other?
- What is fault tolerance, and how does it differ from high availability?
- What is graceful degradation, and how does it improve the user experience during a system failure?
- How do load balancers contribute to high availability in a distributed system?
- What is failover, and how does it help maintain availability during system failure?
- How can health monitoring and self-healing systems help improve system reliability and availability?